

Plan de Sprint 1 — Fundamentos del Sistema

Estado real verificado en el repo antes de planificar: H0001 (Crear torneo) está mayormente implementado desde una sesión anterior — backend completo (endpoints, validación, tests, migración) y frontend completo (formulario, listado). Lo que falta de H0001 es solo la integración con auth real. H0002, autenticación JWT, Dockerfiles de backend/frontend y CI/CD no existen todavía (verificado: no hay .github/workflows, no hay Dockerfile en backend/frontend, no hay entidad Usuario ni endpoint de login).

1. Objetivo del Sprint

Cerrar el ciclo completo de un Coordinador autenticado creando un torneo y definiendo sus categorías, sobre una infraestructura que se pueda levantar con un solo comando (docker-compose up) y que tenga un pipeline de CI que falle si algo se rompe. Al final del sprint, alguien externo al equipo tiene que poder clonar el repo, levantar todo, loguearse y crear un torneo con categorías sin tocar código.

2. Historias de Usuario incluidas

H0001 — Crear torneo (2 pts, Básico) — 85% hecho

Criterios de aceptación:

- Coordinador crea un torneo con nombre, fecha (futura), lugar, flyer opcional, queda en estado Borrador
- Sistema valida fecha futura y campos obligatorios con mensaje específico
- Coordinador ve el listado de torneos creados
- Pendiente: la creación requiere estar logueado como Coordinador (hoy el endpoint exige el rol pero no hay forma de obtener el token)

H0002 — Definir categorías (5 pts, Básico) — no empezado

Criterios de aceptación:

- Coordinador, dentro de un torneo, crea una categoría con: nombre, tipo (Combate/Formas), sexo (M/F), rango de edad (min/max), rango de peso (min/max), rango de graduación (min/max)
- El sistema rechaza rangos inválidos (min > max) con mensaje específico
- Coordinador ve el listado de categorías de un torneo
- Coordinador puede editar una categoría (corrección de rangos antes de que haya competidores cargados)
- Una categoría no se puede eliminar si ya tiene competidores inscriptos

Setup inicial — parcial

Criterios de aceptación:

- docker-compose up --build levanta los 5 servicios (hoy falla en backend/frontend por falta de Dockerfile)
- Pipeline de CI corre en cada push/PR: build + test backend, build + lint frontend
- Variables de entorno documentadas y sin secretos hardcodeados en el repo

Autenticación JWT básica (login/logout) — no empezado

Criterios de aceptación:

- Existe al menos un usuario Coordinador para probar (seed, no historia de registro todavía)
- POST /api/v1/auth/login con email/password devuelve un JWT con claim de rol

Sistema de Gestión de Torneos Deportivos

- El JWT emitido es aceptado por los endpoints existentes de Torneos
- Frontend: pantalla de login, guarda el token (Zustand authStore), redirige al dashboard
- Logout: limpia el token del store y redirige a login

3. Desglose de tareas técnicas

#	Tarea	Área	Estimación	Depende de
1	Entidad Usuario (Domain) + enum RolUsuario	Backend	2h	-
2	UsuarioConfiguration (EF) + migración	Backend	2h	1
3	Seed de 1 usuario Coordinador (hash de password)	Backend	3h	2
4	LoginCommand/Handler: valida credenciales, genera JWT	Backend	4h	3
5	LoginEndpoint (FastEndpoints, AllowAnonymous) + Validator	Backend	2h	4
6	Mover secretos (Jwt, Postgres, MinIO) a variables de entorno	Backend	2h	-
7	Commands/Queries de Categoría (Create/Update/GetByTorneo)	Backend	6h	-
8	Validación de rangos (min <= max) en Categoría	Backend	2h	7
9	Endpoints FastEndpoints de Categorías	Backend	4h	7, 8
10	Tests xUnit de Categorías y de Login	Backend	5h	5, 9
11	Dockerfile multi-stage para TorneoDeportivo.API	Infra	3h	-
12	Dockerfile multi-stage para frontend	Infra	3h	-
13	Ajustar docker-compose.yml con env vars	Infra	2h	11, 12, 6
14	GitHub Actions: workflows de backend y frontend	CI/CD	4h	-
15	Frontend: authStore, pantalla de login, RoleGuard básico	Frontend	6h	5

Sistema de Gestión de Torneos Deportivos

16	Frontend: manejo de 401 (redirigir a login)	Frontend	2h	15
17	Frontend: types + api + hooks de Categorías	Frontend	4h	9
18	Frontend: CategoriaForm, CategoriaList, ruta de categorías	Frontend	8h	17
19	Verificación manual end-to-end con Postgres real vía Docker	QA	3h	9, 13, 18

Total estimado: ~67h (aprox. 1.5 desarrolladores full-time en 2 semanas, dejando margen para imprevistos).

4. Dependencias entre tareas

Usuario (1) -> EF config+migracion (2) -> Seed (3) -> LoginCommand (4) -> LoginEndpoint (5) -> AuthStore+Login UI (15) -> 401 handling (16)	
(10)	-> Tests login
Secrets a env vars (6) -> docker-compose ajustado (13)	
Categoria Commands (7) -> Validacion rangos (8) -> Endpoints Categorias (9) -> Tests categorias (10)	
CategoriaForm/List (18)	-> Frontend api/hooks (17) ->
Dockerfile backend (11) -> docker-compose ajustado (13)	
Dockerfile frontend (12) ->	
[5, 9, 13, 18] -> Verificacion E2E con Docker real (19)	

La cadena crítica es 1→2→3→4→5→15→16, en paralelo con 7→8→9→17→18; ambas convergen en la tarea 19, que cierra el sprint.

5. Definición de Done (DoD) del Sprint

- docker-compose up --build levanta los 5 servicios sin errores, desde cero, en una máquina limpia
- Ningún secreto está hardcodeado en archivos versionados
- CI verde en la rama principal: build+test backend, build+lint frontend
- Un Coordinador puede loguearse, crear un torneo, crear al menos 2 categorías y verlas listadas, probado contra Postgres real
- Cobertura de tests: todo handler nuevo tiene al menos 1 test de happy path + 1 de error
- skills/ux-ui-guidelines.md aplicado en las pantallas nuevas

Sistema de Gestión de Torneos Deportivos

- .env.example actualizado con todas las variables nuevas

6. Riesgos específicos de este sprint

Alto — Autenticación es nueva infraestructura, no una historia chica. Implica entidad nueva, hashing de passwords, emisión/validación de JWT y wiring en todos los endpoints existentes. Mitigación: priorizar las tareas 1-6 en la primera semana.

Medio — Docker no probado en este entorno todavía. Esta máquina de desarrollo no tiene Docker instalado. Mitigación: priorizar la verificación E2E (tarea 19) a mitad de sprint, no al final.

Medio — Secretos hardcodeados hoy en appsettings.json. Mover el secreto de JWT justo cuando se construye el login que depende de él es delicado. Mitigación: hacer el cambio de secrets (tarea 6) antes de tocar el login (tarea 4).

Bajo — Alcance de logout ambiguo. Sin blacklist de tokens ni refresh tokens, logout es solo borrar el token del cliente. Comunicarlo como limitación conocida, no como bug.